

hexens x 1inch"

Security Review Report for 1inch

June 2026



Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
 - Security Review Lead
 - Scope
 - Changelog
4. Severity Structure
 - Severity characteristics
 - Issue symbolic codes
5. Findings Summary
6. Weaknesses
 - Amount-in fee wrapper can override SplineSwap exact-in partial fill
 - One-Sided SplineSwap Orders Are Non-Functional
 - Taker Can Extract a Small Amount of Base Tokens with Zero Quote Input

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This audit reviews an update to 1inch Swap VM, a computation engine that executes token swap strategies from bytecode programs. The update introduces SplineSwap, a composable, multi-region AMM with separated density and pricing layers that can be configured independently for each region.

Our review was conducted over a three-day period and included a comprehensive analysis of the relevant smart contracts.

During the assessment, we identified one medium-severity issue that could cause takers to pay more input tokens when combining SplineSwap with fees. We also identified two low-severity issues.

All identified issues were remediated by the development team and subsequently verified by our auditors.

Following the remediation phase, we conclude that the protocol's overall security posture and code quality have improved as a result of this audit.

3. Security Review Details

- **Review Led by**

Trung Dinh, Lead Security Researcher

- **Scope**

The analyzed resources are located on:

<https://github.com/1inch/swap-vm-private/tree/6bc9a9c1b7c68840b2dde62183cccf6f82ad31f9>

- [src/instructions/SplineSwap.sol](#)
- [src/libs/SplineSwapMath.sol](#)
- [src/opcodes/AquaOpcodes.sol](#)
- [src/routers/AquaSwapVMRouter.sol](#)

The issues described in this report were fixed in the following commit:

<https://github.com/1inch/swap-vm-private/tree/d2b19ef62dd53db095ea962b88aaecb19290e8b7>

- **Changelog**

■ 9 June 2026	Audit start
■ 12 June 2026	Initial report
■ 18 June 2026	Revision received
■ 2 July 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

- 1. Impact of the vulnerability
- 2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

▪ Severity Characteristics

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical	Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.
High	Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium

Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.

Low

Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.

Informational

Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

▪ Issue Symbolic Codes

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity

Number of findings

Critical	0
High	0
Medium	1
Low	2
Informational	0

Total:

3



Medium

Low



Fixed

6. Weaknesses

This section contains the list of discovered weaknesses.

OIN16-1 | Amount-in fee wrapper can override SplineSwap exact-in partial fill

Fixed 

Severity:

Medium

Probability:

Unlikely

Impact:

Medium

Path:

```
src/instructions/SplineSwap.sol#L197  
src/instructions/Fee.sol#L79  
src/instructions/Fee.sol#L237
```

Description:

`Fee._flatFeeAmountInXD` wraps the downstream swap execution by reducing `ctx.swap.amountIn`, calling `ctx.runLoop()`, and then restoring the original taker-defined input amount.

```
if (ctx.query.isExactIn) {  
    // Decrease amountIn by fee only during swap-instruction  
    uint256 takerDefinedAmountIn = ctx.swap.amountIn;  
    ctx.swap.amountIn -= Math.ceilDiv(ctx.swap.amountIn * feeBps, BPS);  
    ctx.runLoop();  
    ctx.swap.amountIn = takerDefinedAmountIn;  
}
```

The same restore pattern is used by `_feeAmountIn`.

```
if (ctx.query.isExactIn) {  
    // Decrease amountIn by fee only during swap-instruction  
    uint256 takerDefinedAmountIn = ctx.swap.amountIn;  
    feeAmountIn = ctx.swap.amountIn * feeBps / BPS;  
    ctx.swap.amountIn -= feeAmountIn;  
    ctx.runLoop();  
    ctx.swap.amountIn = takerDefinedAmountIn;  
}
```

SplineSwap._splineSwap2D supports exact-in partial fills by reducing **ctx.swap.amountIn** when the requested base amount exceeds the remaining curve capacity.

```
(uint256 scaleIn, uint256 scaleOut) = tokenInIsLt
? (uint256(cfg.scaleLt), uint256(cfg.scaleGt))
: (uint256(cfg.scaleGt), uint256(cfg.scaleLt));

if (ctx.query.isExactIn) {
    require(ctx.swap.amountOut == 0, SplineSwapRecomputeDetected());

    // preprocessing - normalize amounts/balances into internal units.
    uint256 baseBalance = ctx.swap.balanceIn * scaleIn;
    uint256 quoteBalance = ctx.swap.balanceOut * scaleOut;
    require(baseBalance > 0 && quoteBalance > 0,
SplineSwapRequiresBothBalancesNonZero(baseBalance, quoteBalance));
    uint256 requestedBase = ctx.swap.amountIn * scaleIn;
    uint256 baseAmount = requestedBase;
    uint256 totalCapacity = cfg.sellCapacity + cfg.buyCapacity;
    uint256 baseRoom = baseBalance < totalCapacity ? totalCapacity - baseBalance : 0;
    if (baseAmount > baseRoom) baseAmount = baseRoom;

    // quote calculation
    (bool startIsSell, uint256 x0) = _resolveZone(cfg, baseBalance);
    (uint256 quote, uint256 baseConsumed) = _quoteFromBase(cfg, x0, baseAmount, startIsSell, true);

    // postprocessing - denormalize.
    require(quote <= quoteBalance, SplineSwapRecomputeDetected());
    ctx.swap.amountOut = quote / scaleOut;
    if (baseConsumed < requestedBase) ctx.swap.amountIn = Math.ceilDiv(baseConsumed, scaleIn);
}
```

When an amount-in fee instruction is placed before **SplineSwap**, this partial-fill adjustment is overwritten after **ctx.runLoop()** returns. As a result, settlement can use the original exact-in amount even though **SplineSwap** only priced and returned output for the capacity-clamped amount. The taker can therefore pay more input than the amount actually priced by **SplineSwap**.

Remediation:

- Preserve downstream changes to **ctx.swap.amountIn** after **ctx.runLoop()** instead of unconditionally restoring the original taker-defined amount.
- Apply the amount-in fee relative to the final consumed input amount when downstream execution performs a partial fill.

OIN16-2 | One-Sided SplineSwap Orders Are Non-Functional

Fixed ✓

Severity:

Low

Probability:

Rare

Impact:

Medium

Path:

src/instructions/SplineSwap.sol#L184

src/instructions/SplineSwap.sol#L205

Description:

SplineSwap supports a sell zone (maker sells base) and/or a buy zone (maker buys base). However, **_splineSwap2D** enforces a check that requires both balances to be nonzero, which is impossible for any one-sided order. For one-sided orders, the intended trade direction always reverts

A maker creates a **SplineSwap** order by choosing a sell capacity, a buy capacity, or both:

```
require(args.sellCapacity > 0 || args.buyCapacity > 0, SplineSwapInvalidCapacities(...));
```

A sell-only maker (**buyCapacity = 0**) is instructed to deposit base and no quote. A buy-only maker (**sellCapacity = 0**) deposits quote and no base.

So during **_splineSwap2D** both the **ExactIn** and **ExactOut** flows have the same guard

```
require(baseBalance > 0 && quoteBalance > 0, SplineSwapRequiresBothBalancesNonZero(...));
```

- Sell-only order → **ExactOut** (taker buys base) reverts because **quoteBalance = 0**
- Buy-only order → **ExactIn** (taker sells base) reverts because **baseBalance = 0**

Remediation:

ExactOut: only **baseBalance** matters, the maker delivers base, quote balance is irrelevant.

```
-- require(baseBalance > 0 && quoteBalance > 0, ...);  
++ require(baseBalance > 0, ...);
```

ExactIn: only **quoteBalance** matters, the maker delivers quote.

```
-- require(baseBalance > 0 && quoteBalance > 0, ...);  
++ require(quoteBalance > 0, ...);
```

OIN16-3 | Taker Can Extract a Small Amount of Base Tokens with Zero Quote Input

Fixed 

Severity:

Low

Probability:

Rare

Impact:

Medium

Path:

src/libs/SplineSwapMath.sol#L195

Description:

The function `SplineSwapMath.quoteOf()` is used to calculate the quote amount required to move across a zone slice `[lo, hi]`.

```
function quoteOf(
    uint8 pairId,
    uint16 rangeBps,
    uint256 lo,
    uint256 hi,
    uint256 cap,
    uint256 initialPrice,
    uint16 spreadBps,
    bool zoneIsSell,
    bool isSell
) internal pure returns (uint256) {
    if (lo == hi) return 0;

    uint256 dTau = tauOf(pairId, rangeBps, hi, zoneIsSell)
        - tauOf(pairId, rangeBps, lo, zoneIsSell);

    uint256 mid = Math.mulDiv(
        dTau,
        cap * initialPrice,
        ONE * ONE,
        isSell ? Math.Rounding.Floor : Math.Rounding.Ceil
    );

    return _applySpread(mid, spreadBps, isSell);
}
```

The issue arises in the **dTau** calculation, where it can evaluate to **0** even when **lo != hi**. As a consequence, a taker can extract some base tokens without providing any quote tokens as input. This issue occurs in an edge case when buying base tokens and **lo** and **hi** are very close to **ONE**. In this region, **tau** gradually decreases as it approaches **ONE**, and rounding can cause both endpoints to produce the same **tau** value, resulting in **dTau = 0**.

Furthermore, in some cases **dTau** can become negative. In those cases, the transaction simply reverts and no further impact occurs.

Remediation:

Consider reverting the function when **hi != lo** and **dTau == 0**.

Proof of Concept:

```
// SPDX-License-Identifier: LicenseRef-Degensoft-SwapVM-1.1
pragma solidity 0.8.30;

import { Test, stdError } from "forge-std/Test.sol";

import { SplineSwapMath } from "../src/libs/SplineSwapMath.sol";
import { Power } from "../src/libs/Power.sol";

import "forge-std/console.sol";

contract SplineSwapMathHarness {
    function calculateZoneSwap(
        uint256 x0, uint256 x1, uint256 cap, uint256 rangeBps,
        bytes4 density, bytes4 price, uint256 spreadBps,
        bool isSellZone, uint256 initialPrice
    ) external pure returns (uint256, uint256) {
        return SplineSwapMath.calculateZoneSwap(
            x0, x1, cap, rangeBps, density, price, spreadBps, isSellZone, initialPrice
        );
    }

    function quoteOf(
        uint8 pairId, uint16 rangeBps, uint256 lo, uint256 hi,
        uint256 cap, uint256 initialPrice, uint16 spreadBps,
        bool zonelsSell, bool isSell
    ) external pure returns (uint256) {
        return SplineSwapMath.quoteOf(
            pairId, rangeBps, lo, hi, cap, initialPrice, spreadBps, zonelsSell, isSell
        );
    }
}

contract SplineSwapMathMonotonicityTest is Test {

    uint256 constant ONE = SplineSwapMath.ONE;

    uint256 constant X_LO = ONE - 1;
    uint256 constant X_HI = ONE;

    uint8 constant PAIR_ID = SplineSwapMath.PAIR_CBRT_SPLINE;
    uint16 constant RANGE_BPS = 10000;
```

```
SplineSwapMathHarness harness = new SplineSwapMathHarness();
```

```
function test_quoteOf_panics() public {  
    vm.expectRevert(stdError.arithmeticError);  
    harness.quoteOf(  
        PAIR_ID,  
        RANGE_BPS,  
        X_LO - 1e8,  
        X_HI,  
        1e24,  
        SplineSwapMath.ONE,  
        0,  
        false,  
        true  
    );  
}
```

```
function test_quoteOf_zero_quote_for_nonzero_interval() public {  
    uint256 lo = ONE - 2_000_000_000;  
    uint256 hi = ONE - 1_000_000_000;  
  
    uint256 tauLo = SplineSwapMath.tauOf(PAIR_ID, RANGE_BPS, lo, false);  
    uint256 tauHi = SplineSwapMath.tauOf(PAIR_ID, RANGE_BPS, hi, false);  
  
    assertEq(tauLo, tauHi, "both endpoints share the same tau value in the plateau");  
    assertLt(lo, hi, "interval is non-empty");  
  
    uint256 cap = 1e24;  
  
    uint256 quote = harness.quoteOf(  
        PAIR_ID,  
        RANGE_BPS,  
        lo,  
        hi,  
        cap,  
        ONE,  
        0,  
        false,  
        false  
    );  
  
    uint256 baseReceived = (hi - lo) * cap / ONE;
```

```
console.log("baseReceived = ", baseReceived);  
console.log("quote =", quote);  
  
assertGt(baseReceived, 0, "non-zero base transferred");  
assertEq(quote, 0, "taker pays 0 quote for real base tokens");  
}  
}
```

hexens x 1inch"

